

Requintu

Documentación Técnica del Proyecto

Plataforma web + app Android de negocios locales de Colombia

Backend: Node.js / Express — Frontend: React / Vite

Base de datos: MySQL (TiDB Cloud) — Imágenes: Cloudinary

Tiempo real: Server-Sent Events — Correo: Brevo (cola con reintentos)

Repositorio: github.com/aycardy93-eng/-requintu

Versión 2.0 — 28 de agosto de 2026

1. Resumen del proyecto

Requintu es una aplicación que conecta a viajeros y usuarios con los negocios locales de Colombia. Los usuarios pueden explorar locales a través de un **mapa interactivo** dividido por departamentos y municipios, consultar el detalle de cada negocio (fotos, promociones, calificaciones, ubicación), publicar contenido en un **muro comunitario con avisos en tiempo real** y, en el caso de los comerciantes, crear y administrar sus propios locales. Una cuenta de administrador permite gestionar usuarios, locales y publicaciones en un panel dedicado.

El proyecto está formado por dos clientes y un solo backend:

- **Web** (SPA React) desplegada en Vercel.
- **App nativa Android** (Capacitor 8) que empaqueta el mismo frontend (firmada en release para Google Play).
- **API REST** en Node.js/Express desplegada en Render.

2. Arquitectura general

El sistema sigue una arquitectura de **cliente-servidor** con API REST y autenticación por tokens JWT, más un canal **Server-Sent Events (SSE)** para avisar en vivo de las novedades del muro:



- **Frontend:** React 19 con React Router 7 (SPA). Compilado con Vite 8; se sirve como estático en Vercel con rewrites SPA (configuración en frontend/vercel.json, Root Directory frontend).
- **App nativa:** Capacitor 8 genera el proyecto Android en frontend/android; el WebView carga el mismo dist. Firmada en release (keystore propio) para Google Play.
- **Backend:** Express 4 (módulos ES). Expone /api/*, sirve /uploads y aplica seguridad (helmet, rate limit, CORS, HTTPS). Incluye un canal SSE y una cola de correos persistida.
- **Base de datos:** MySQL 8 administrada (TiDB Cloud) vía mysql2 (pool de conexiones, TLS).
- **Almacenamiento:** las imágenes se suben al backend con Multer (con validación de formato real) y se publican en Cloudinary.
- **Correo:** la recuperación de contraseña se encola (tabla cola_emails) y se envía con la API v3 de Brevo, con reintentos y backoff.
- **Tiempo real:** el muro de publicaciones recibe eventos *nueva*, *editada* y *borrada* por SSE sin dependencias externas.

3. Stack tecnológico

3.1 Backend (raíz del repositorio)

Tecnología	Uso
Node.js (ESM)	Entorno de ejecución. <code>npm start</code> ejecuta <code>index.js</code> .
Express 4	Servidor HTTP y enrutado REST.
mysql2	Pool de conexiones a MySQL / TiDB Cloud.
jsonwebtoken + bcryptjs	Access tokens JWT (1 h) y hash de contraseñas (bcrypt, 10 rondas).
cookie-parser	Lectura de la cookie <code>httpOnly</code> de refresh token.
multer	Recepción de archivos (solo imágenes JPEG/PNG/WebP, máx. 5 MB, validados por contenido).
cloudinary (SDK 2)	Publicación de imágenes en la nube.
express-validator	Validación y saneamiento de entradas.
express-rate-limit	Límite de peticiones por IP (API y autenticación).
helmet	Cabeceras de seguridad HTTP (HSTS, CORS cross-origin para recursos).
cors	Lista blanca de orígenes (web + Capacitor).
Server-Sent Events (nativo)	Canal GET <code>/api/eventos</code> para avisos del muro (sin dependencias).
node:test	Suite de pruebas de integración (<code>npm test</code> , 21 pruebas).
dotenv	Variables de entorno.

3.2 Frontend (carpeta `frontend`)

Tecnología	Uso
React 19 + Vite 8	SPA y empaquetado de producción.
react-router-dom 7	Enrutado del cliente.
Capacitor 8 (core, cli, android, app)	Empaquetado nativo Android y control del botón «atrás».
EventSource (API web)	Suscripción del muro a los avisos en tiempo real.
swiper	Carrusel de imágenes (login e inicio).
oxlint	Linter (<code>npm run lint</code>).
Playwright (@playwright/test)	Pruebas e2e (9 pruebas) en Chromium headless.
sharp	Dev: generación de iconos de la app.

4. Estructura del repositorio

```
proyecto/
├─ index.js           # API principal (Express, rutas REST + SSE)
├─ db.js             # Pool de conexiones MySQL + health check
├─ emailQueue.js      # Cola de correos con reintentos y backoff (Brevo)
├─ filtroProfano.js   # Filtro de groserías (leetspeak, repetidos)
├─ .env.example       # Plantilla de variables de entorno
├─ requintu_dump.sql  # Respaldo del esquema/datos (incluye cola_emails)
├─ netlify.toml       # Alternativa de despliegue (Netlify)
├─ package.json       # Dependencias del backend
├─ test/
│   └─ api.test.mjs   # 21 pruebas de integración (node:test)
├─ Documentacion_Tecnica_Requintu.pdf|docx
├─ Manual_de_Uso_Requintu.pdf / Informe_de_Pruebas_Requintu.pdf
├─ Requintu-app-release.aab|apk / Requintu-app-debug.apk (builds firmados)
└─ frontend/
    ├─ capacitor.config.json # Config de Capacitor (appId, webDir)
    ├─ index.html           # HTML base
    ├─ vercel.json          # Config de Vercel (framework, rewrites SPA)
    ├─ vite.config.js
    ├─ playwright.config.js # E2E: backend local + preview de producción
    ├─ e2e/                 # specs: inicio, privacidad, auth, muro
    └─ public/
        ├─ sw.js           # service worker auto-destructivo (PWA desactivada)
        └─ politica-de-privacidad.html
    └─ android/            # proyecto Capacitor (firma release en app/build.gradle)
    └─ src/
        ├─ main.jsx        # Punto de entrada (React Router + AuthProvider)
        ├─ App.jsx         # Rutas de la SPA
        ├─ config.js       # API_URL según VITE_BACKEND_ORIGIN
        ├─ context/AuthContext.jsx # Sesión: login, logout, refresco automático
        ├─ components/     # Navbar, ProtectedRoute, AdminRoute, InstallBanner, FondoPagina
        ├─ pages/          # Home, Login, Register, MapaColombia, Locales,
        │                  # LocalDetalle, CrearLocal, Publicaciones, Perfil,
        │                  # Admin, OlvidePassword, ResetPassword, PoliticaPrivacidad
        └─ assets/         # co-departamentos.svg, imágenes
```

5. Backend

5.1 Punto de entrada (`index.js`)

Configura Express, aplica los middlewares globales y registra más de **30 rutas** bajo `/api`. En producción fuerza HTTPS (redirect 301), confía en el proxy (`trust proxy`), activa HSTS y usa cookies `SameSite=None; Secure` para permitir el login desde la app nativa. Al arrancar verifica la conexión a la BD y levanta la cola de correos (`iniciarColaCorreos()`).

5.2 Conexión a la base de datos (`db.js`)

Crea un pool con `mysql2/promise` (límite de 5 conexiones por defecto, `keep-alive`). Si `DB_SSL=true` (exigido por TiDB Cloud) activa TLS. Exporta `checkDatabaseHealth()` que valida la conexión con `SELECT 1`.

5.3 Filtro de groserías (`filtroProfano.js`)

Normaliza el texto (minúsculas, leetspeak `0-o, 3-e, 4-a, @-a...`, caracteres repetidos: `joder` → `joder`) y lo compara contra una lista en español. Evita falsos positivos en palabras cortas (longitud mínima). Se aplica con el middleware `sinGroserias(campos)` a locales, planes, calificaciones y publicaciones.

5.4 Autenticación y autorización

- **Registro:** valida nombre, correo (único) y contraseña (mín. 8). Roles permitidos: *turista* (por defecto) y *comerciante*. Contraseña cifrada con `bcrypt`.
- **Login:** devuelve un **access token JWT (1 h)** con `{ id, nombre, rol }` y emite una **cookie httpOnly** con el refresh token.
- **Refresh token:** generado con `crypto.randomBytes(64)`, se guarda *hasheado* (SHA-256) en la tabla `refresh_tokens` con expiración de 7 días. Cada refresco **rota** el token (se revoca el anterior). La cookie se limita a la ruta `/api/auth`.
- **Logout:** revoca el refresh token y limpia la cookie.
- **Protección:** `authMiddleware` verifica el JWT (cabecera `Authorization: Bearer`) y rellena `req.user`; `checkRole([...])` restringe acciones a roles concretos (p. ej. crear locales requiere *comerciante* o *admin*).

5.5 Recuperación de contraseña (Brevo vía cola)

`/api/forgot-password` genera un token (64 hex, expira en 1 h) guardado *hasheado* en `password_resets`. El correo con el enlace se **encola** en la tabla `cola_emails` y un procesador lo envía con la **API v3 de Brevo** (`api.brevo.com/v3/smtp/email`) con reintentos y backoff; la respuesta de la API no se bloquea esperando el envío. `/api/reset-password` valida el token (un solo uso), actualiza la contraseña, marca el reset como usado y revoca las sesiones previas.

5.6 Avisos en tiempo real del muro (SSE)

- **Endpoint:** GET `/api/eventos` (público, Content-Type: `text/event-stream`). El frontend se suscribe con **EventSource** y el servidor mantiene la conexión abierta.
- **Eventos emitidos:** nueva (id, usuario_id y autor al crear una publicación), editada y borrada (id). Incluye la moderación del admin.
- **Heartbeat:** cada 25 s se envía un comentario : `latido` para mantener viva la conexión en proxies (p. ej. Render) y detectar clientes caídos.
- **Publicación (broadcast):** los clientes suscritos se guardan en un set en memoria; cada evento se serializa y se escribe a todas las conexiones.
- No requiere dependencias nuevas (usa el `res` de Express) y respeta CORS de la API.

5.7 Seguridad

Medida	Implementación
Cabeceras HTTP	helmet + HSTS (365 días) + redirect obligatorio a HTTPS en producción.
CORS	Lista blanca (local, Vercel y Capacitor: <code>capacitor://localhost</code>).

Rate limiting	300 req / 15 min en /api; 10 req / 15 min en login y registro.
Validación de entrada	express-validator en todas las rutas de escritura.
Contenido inapropiado	Middleware sinGroserias.
Subida de archivos	Solo <code>image/jpeg png webp</code> ; extensión, MIME y magic bytes comprobados (400 si no es una imagen real), máx. 5 MB.
Contraseñas	bcrypt (10 rondas); nunca en texto plano.
Tokens	JWT firmado con JWT_SECRET; refresh tokens hasheados en BD y rotados.
Correo	Cola persistida con reintentos, backoff exponencial y registro del error en BD.

5.8 Subida de imágenes

POST /api/upload (protegido) recibe la imagen con Multer, comprueba sus **magic bytes** (JPEG, PNG o WebP) rechazando archivos que no sean imágenes reales (respuesta 400), la sube a **Cloudinary** mediante el SDK `uploader.upload` (carpeta según entidad) y devuelve la URL pública. El frontend usa esta URL para persistir locales, planes y publicaciones.

5.9 Cola de correos con reintentos (`emailQueue.js`)

- **Persistencia:** tabla `cola_emails` (destinatario, asunto, HTML, estado *pendiente/enviado/fallido*, intentos, próximo intento, error). La tabla se crea automáticamente (`CREATE TABLE IF NOT EXISTS`).
- **Encaje:** `encolarCorreo()` inserta la fila y dispara el procesamiento; nunca lanza. Si la BD no responde, el correo se conserva en memoria.
- **Procesamiento:** un worker envía por Brevo los pendientes cuyo *próximo_intento* ya venció. Al fallar, incrementa intentos y programa el siguiente con **backoff exponencial** (1, 2, 4 y 8 min). Tras 5 intentos marca *fallido* y registra el error.
- **Resiliencia:** al iniciar el servidor (`iniciarColaCorreos()`) se reanudan los correos que quedaron pendientes (sobreviven reinicios), con escaneo automático cada 15 s.

6. API REST (resumen de endpoints)

6.1 Autenticación y perfil

Método	Ruta	Acceso	Descripción
POST	/api/register	Público	Registro (turista/comerciante).
POST	/api/login	Público	Inicio de sesión (JWT + cookie refresh).
POST	/api/auth/refresh	Cookie	Renueva access token (rota refresh).
POST	/api/auth/logout	Cookie	Cierra sesión.
POST	/api/forgot-password	Público	Encola el enlace de recuperación (Brevo).
POST	/api/reset-password	Público	Reestablece contraseña con token de 1 uso.
GET	/api/perfil	Autenticado	Datos del perfil.

6.2 Catálogo

Método	Ruta	Acceso	Descripción
GET	/api/municipios	Público	Lista de municipios.
GET	/api/categorias	Público	Categorías de locales.
GET	/api/departamentos	Público	Departamentos con datos para el mapa.
GET	/api/departamentos/:nombre/municipios	Público	Municipios de un departamento.
GET	/api/eventos	Público	Canal SSE de avisos del muro (<i>nueva/editada/borrada</i>).

6.3 Locales, calificaciones y planes

Método	Ruta	Acceso	Descripción
GET	/api/locales	Público	Listado con filtros y búsqueda.
POST	/api/locales	Comerciante/Admin	Crea un local (valida unicidad por municipio).
GET	/api/locales/:id	Público	Detalle con calificación promedio.
PUT	/api/locales/:id	Autenticado	Edita (dueño/admin; evita local repetido).
DELETE	/api/locales/:id	Autenticado	Elimina (dueño/admin).
GET	/api/locales/:id/calificaciones	Público	Reseñas del local.
POST	/api/locales/:id/calificaciones	Autenticado	Puntúa (1-5) y comenta.
GET	/api/locales/:id/planes	Público	Planes/paquetes del local.
POST	/api/locales/:id/planes	Autenticado	Agrega plan.
PUT / DELETE	/api/planes/:id	Autenticado	Edita / elimina un plan.

6.4 Subida y publicaciones

Método	Ruta	Acceso	Descripción
POST	/api/upload	Autenticado	Sube imagen a Cloudinary (valida formato real).
GET	/api/publicaciones	Público	Muro de publicaciones.
POST	/api/publicaciones	Turista/Admin	Publica contenido (emite evento SSE <i>nueva</i>).
PUT / DELETE	/api/publicaciones/:id	Autenticado	Edita / elimina (autor/admin; emite <i>editada/borrada</i>).

6.5 Administración (solo rol *admin*)

Método	Ruta	Descripción
GET	/api/admin/estadisticas	Totales y distribución por rol.
GET	/api/admin/usuarios	Listado con paginación y rol.
PUT	/api/admin/usuarios/:id/rol	Cambia el rol de un usuario.
DELETE	/api/admin/usuarios/:id	Elimina un usuario.
GET	/api/admin/locales	Listado paginado de locales.
GET	/api/admin/publicaciones	Listado paginado de publicaciones.
DELETE	/api/admin/publicaciones/:id	Modera (elimina) una publicación (emite SSE <i>borrada</i>).

7. Base de datos

MySQL 8 (TiDB Cloud). Esquema principal exportado en requintu_dump.sql:

Tabla	Contenido	Claves principales
usuarios	Nombre, email único, password_hash, rol (admin/comerciante/turista), foto_perfil, reset_token y expiración.	PK id_usuario; UNIQUE email
municipios	~1120 municipios de Colombia con su departamento.	PK id_municipio
categorias	13 categorías (restaurante, café, alojamiento, turismo, artesanías, etc.).	PK id_categoria
locales	Nombre, descripción, dirección, teléfono, imagen, FK a categoría, municipio y propietario.	FKs con ON DELETE SET NULL
calificaciones	Reseñas: puntuación (1-5 con CHECK), comentario, fecha; usuario y local.	FKs con ON DELETE CASCADE
planes	Planes/paquetes del local: título, descripción, precio, fechas, imagen.	FK id_local
publicaciones	Contenido del muro, imagen opcional, fecha, autor.	FK usuario_id
refresh_tokens	Refresh tokens (SHA-256), expiración, revocado, creado_en.	UNIQUE token_hash
password_resets	Tokens de recuperación (hash, expiración, usado). Se crea en producción por las rutas de contraseña.	FK id_usuario
cola_emails	Cola de correos: destinatario, asunto, HTML, estado (pendiente/enviado/fallido), intentos, próximo intento y error.	PK id; INDEX (estado, proximo_intento)
posts	Tabla heredada de una iteración anterior (sin uso activo en la UI).	FK id_usuario

Nota: en los INSERT del respaldo los acentos aparecen con caracteres de escape; las tablas usan juego de caracteres utf8mb4.

8. Frontend

8.1 Inicio y rutas (main.jsx, App.jsx)

main.jsx monta React 19 con `BrowserRouter` y `AuthProvider`. App.jsx define las rutas públicas (`/`, `/login`, `/register`, `/olvide-password`, `/reset-password`, `/politica-de-privacidad`), las de lectura (`/publicaciones`, `/locales`, `/locales/:id`) y las protegidas (`/mapa`, `/crear-local`, `/perfil` con `ProtectedRoute`; `/admin` con `AdminRoute`). También registra un listener del **botón atrás** de Android vía Capacitor (vuelve atrás o cierra la app).

8.2 Gestión de sesión (context/AuthContext.jsx)

- El **access token vive en memoria** (estado React); **nunca** se guarda en localStorage ni en cookies legibles por JavaScript.
- Al montar la app se llama `/api/auth/refresh` con la cookie `httpOnly` para restaurar la sesión.
- Un temporizador renueva el token **5 minutos antes** de la expiración (exp del JWT), evitando sesiones caídas.
- **Logout** limpia el estado y revoca el token en el servidor.
- El rol (admin/comerciante/turista) se decodifica del JWT y condiciona menús y rutas.

8.3 Pantallas principales

Página	Función
Home	Portada con carrusel (swiper) y accesos a secciones.
Login / Register	Acceso con carrusel decorativo y diseño responsive (Login.css).
OlvidePassword / ResetPassword	Flujo de recuperación de contraseña.
MapaColombia	Mapa SVG de departamentos (assets/co-departamentos.svg) con zoom/pan propio (Botones +/–, arrastre, doble toque). Al tocar un departamento se muestran sus municipios en un panel flotante y se navega a los locales del municipio.
Locales	Filtros por departamento, municipio y categoría; tarjetas con estrellas y promociones.
LocalDetalle	Fotos, descripción, mapa de Google, planes/promociones, galería de calificaciones y edición (dueño).
CrearLocal	Formulario + vista previa del mapa; alta de local.
Publicaciones	Muro de publicaciones con imágenes y autor. Se suscribe a <code>/api/eventos</code> (EventSource): si llega una publicación nueva muestra el aviso «Hay N publicaciones nuevas en el muro» con un botón Ver ahora que actualiza la lista; las ediciones y borrados de otros usuarios se sincronizan solos. Las publicaciones propias no generan el aviso.
PoliticaPrivacidad	Política de privacidad (también disponible como HTML estático en <code>/politica-de-privacidad.html</code>).
Perfil	Datos personales y edición.
Admin	Panel: resumen de estadísticas y pestañas Usuarios/Locales/Publicaciones con paginación. Las tablas son adaptables (scroll horizontal en móvil).

8.4 Configuración de entorno

config.js lee `VITE_BACKEND_ORIGIN` (variable de compilación) y construye `API_URL`; si no está definida usa `http://localhost:3000`. La conexión SSE usa el mismo origen (`BACKEND_ORIGIN`). En la app nativa se compila apuntando a `https://requintu-1.onrender.com`.

9. App nativa Android (Capacitor)

- **Configuración:** frontend/capacitor.config.json con appld `com.requintu.app`, appName Requintu y webDir `dist`.
- **Proyecto nativo:** generado con `npx cap add android`; el web build se copia con `npx cap sync android`.
- **Compilación:** Gradle 8.14.3 (JDK 21), compileSdk/targetSdk 36, minSdk 24. Genera app-debug.apk (firmado en debug, instalable) y los artefactos release.
- **Firma release:** keystore propio (frontend/android/requintu-release.jks, alias `requintu`) leído desde keystore.properties por app/build.gradle; certificado de subida upload_certificate.der. Ambos están excluidos de git.
- **Artefactos entregables:** en la raíz del repositorio: Requintu-app-release.aab (para Google Play, verificado con apksigner), Requintu-app-release.apk, Requintu-app-debug.apk e iconos para la ficha de Play (Requintu-icóno-play-512.png, Requintu-feature-graphic-1024x500.png).
- **Marca:** iconos propios en todas las densidades y fondo #12283D.
- **Interacción nativa:** botón «atrás» gestionado con @capacitor/app.
- **Backend compatible:** CORS admite `capacitor://localhost` y las cookies en producción usan `SameSite=None` para funcionar dentro del WebView.

10. Despliegue

10.1 Backend (Render)

- URLs: `https://requintu-1.onrender.com` (principal) y `https://requintu.onrender.com`.
- Comando de inicio: `npm start`. Requiere `NODE_ENV=production` y las variables de entorno según `.env.example`.
- El proceso valida `JWT_SECRET` al arrancar, verifica la conexión a la BD y levanta la cola de correos.
- Soporta conexiones SSE (heartbeat de 25 s para mantenerlas vivas en el proxy).

10.2 Frontend (Vercel)

- URL: `https://requintu.vercel.app`. Se despliega desde la rama `main` de GitHub (push automático).
- Configuración en `frontend/vercel.json`; en Vercel el **Root Directory** se apunta a `frontend` (donde vive el `package.json` con el script `build`). Incluye los rewrites de la política de privacidad y el wildcard SPA `/(.*) → /index.html`.
- Variable de compilación: `VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com`.
- Nota: la PWA (service worker) se desactivó a propósito (SW auto-destructivo) para que la web cargue siempre la versión más reciente.

10.3 Netlify (alternativa)

`netlify.toml` define el build en `frontend` y un redirect SPA `/* → /index.html`.

10.4 Base de datos (TiDB Cloud)

MySQL en la nube, host `gateway01.us-east-1.prod.aws.tidbcloud.com`, puerto 4000, con SSL configurado `rejectUnauthorized: true`.

11. Variables de entorno

```
NODE_ENV=production|development
JWT_SECRET=<secreto largo>
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_SSL
CORS_ORIGINS=<orígenes extra separados por coma>
RATE_LIMIT_API_MAX, RATE_LIMIT_AUTH_MAX
BREVO_API_KEY, BREVO_SENDER_EMAIL
EMAIL_MAX_INTENTOS=<reintentos de la cola de correos; por defecto 5>
APP_URL=<URL pública del frontend>
PORT
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET
VITE_BACKEND_ORIGIN=<frontend, en compilación>
```

Advertencia: el archivo .env real no debe subirse a git (está ignorado).

12. Ejecución local

12.1 Backend

```
npm install
cp .env.example .env # completar valores (BD, JWT, Cloudinary, Brevo)
npm run dev           # o npm start
npm test              # 21 pruebas de integración (usa la BD del .env)
```

12.2 Frontend

```
cd frontend
npm install
npm run dev           # Vite (puerto 5173)
npm run lint          # oxlint
npm run build         # compilación de producción
npx playwright install chromium # solo la primera vez (para e2e)
npm run test:e2e      # 9 pruebas e2e (levanta backend local + preview)
```

12.3 App Android

```
cd frontend
VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com npm run build
npx cap sync android
cd android
JAVA_HOME=<jdk 21> ANDROID_HOME=<sdk> ./gradlew assembleDebug
./gradlew bundleRelease # o assembleRelease (usa keystore.properties)
# APK debug en android/app/build/outputs/apk/debug/app-debug.apk
# AAB en android/app/build/outputs/bundle/release/...
```

13. Seguridad y buenas prácticas aplicadas

- Contraseñas con bcrypt y tokens JWT con expiración corta, + refresh tokens rotatorios en cookie httpOnly (mitigan robo y reuso).
- Cabeceras de seguridad (helmet/HSTS), HTTPS forzado y CORS por lista blanca.
- Rate limiting por IP en API y autenticación.
- Validación de entradas y saneamiento en todas las rutas de escritura.
- Filtro de lenguaje inapropiado para contenido generado por usuarios.
- Restricción de subidas de archivos por formato real (magic bytes), MIME y tamaño.
- Reintentos y backoff en el envío de correos mediante cola persistida (no se pierden correos ante fallos temporales de Brevo).
- Variables sensibles únicamente en variables de entorno (nunca en el repositorio).
- Verificación de unicidad del nombre de un local dentro de su municipio.

14. Pruebas automatizadas

14.1 Backend — integración (node:test)

`npm test` (raíz) ejecuta `test/api.test.mjs`: 21 pruebas que levantan la API contra la base de datos real y verifican autenticación, roles, CORS, CRUD de publicaciones, filtros de locales, validación de imágenes (magic bytes), recuperación de contraseña y la rotación/revocación de sesiones. El detalle de cada caso está en el *Informe de Pruebas*.

14.2 Frontend — e2e (Playwright)

`npm run test:e2e` (en frontend) compila el bundle de producción, levanta el backend local y el servidor de preview, y ejecuta 9 pruebas en Chromium headless: carga de la home, buscador de locales, muro, política de privacidad (con y sin extensión), rutas protegidas, registro/login/logout y el aviso en tiempo real del muro.

15. Mejoras futuras sugeridas

- Publicar la app en Google Play (subir el `Requintu-app-release.aab`, registros de la ficha, cuestionarios de contenido y datos).
- Moderación manual de locales antes de publicarlos (flujo de aprobación).
- Migraciones versionadas del esquema (hoy se respalda con dump).
- Panel de la cola de correos en el admin (ver pendientes, intentos y fallidos).
- Notificaciones push nativas (FCM) para la app Android.

16. Glosario de rutas del servidor

- `/api/*` — API REST y canal SSE (`/api/eventos`).
- `/uploads/*` — archivos estáticos locales (legacy de imágenes).
- Cualquier otra ruta en web la resuelve el SPA (rewrites de Vercel/Netlify).

Fin de la documentación técnica.